from the server as otherTickets or myTickets based on whether the status is purchased or held.

Then we have a series of types for each dispatch action, which uses some new TypeScript. It looks like this:

chapter_11/01/app/javascript/contexts/venue_types.ts

```
interface SetTicketToBuy {
  type: "setTicketsToBuy"
  amount: number
}

interface HoldTicket {
  type: "holdTicket"
  seatNumber: number
  rowNumber: number
}

interface UnholdTicket {
  type: "unholdTicket"
  seatNumber: number
  rowNumber: number
}

interface ClearHolds {
  type: "clearHolds"
}

interface SetTickets {
  type: "setTickets"
  tickets: TicketData[]
}

export type VenueAction =
  | ClearHolds
  | HoldTicket
  | SetTicketToBuy
  | SetTickets
  | UnholdTicket
```

The problem we need TypeScript to solve with these action types is that this reducer has multiple action objects, and each of those is going to have a type